

Assignment 17: Employee Management System (Salesforce Apex Console-Based)

Step 1: Create Custom Object

Go to: **Setup** → **Object Manager** → **Create** → **Custom Object**

Create object with:

- **Label:** Employee
- **Plural Label:** Employees
- **Object Name:** Employee
- **Record Name:** Employee Name
- **Data Type:** Text

Click **Save**.

Step 2: Create Custom Fields

Inside the Employee object, create these fields:

1. Emp ID

- Data Type: Number
- Field Label: Emp ID
- Length: 10
- Decimal Places: 0
- API Name: Emp_ID__c

2. Email

- Data Type: Email
- API Name: Email__c

3. Birth Date

- Data Type: Date
- API Name: Birth_Date__c

4. Department

- Data Type: Text
- API Name: Department__c

Default Salesforce field:

- Employee Name → Name

Step 3: Create Apex Class

Go to: **Setup** → **Apex Classes** → **New**

Class Name:

Paste code:

```
public class EmployeeSystem {

    public static void menu(Integer choice) {
        if(choice == 1) {
            createEmployee(
                101,
                'Rahul Sharma',
                'rahul@gmail.com',
                Date.newInstance(2002, 5, 10),
                'IT'
            );
        }
        else if(choice == 2) {
            viewEmployees();
        }
        else if(choice == 3) {
            updateEmployee(101, 'HR');
        }
        else if(choice == 4) {
            deleteEmployee(101);
        }
        else {
            System.debug('Invalid Choice');
        }
    }

    public static void createEmployee(Integer empId,
                                      String empName,
                                      String email,
                                      Date birthDate,
                                      String department) {

        Employee__c emp = new Employee__c();

        emp.Emp_ID__c = empId;
        emp.Name = empName;
        emp.Email__c = email;
        emp.Birth_Date__c = birthDate;
        emp.Department__c = department;

        insert emp;
    }
}
```

```

        System.debug('Employee created successfully');
    }

    public static void viewEmployees() {
        List<Employee__c> empList = [
            SELECT Id, Emp_ID__c, Name, Email__c, Birth_Date__c,
Department__c
            FROM Employee__c
        ];

        for(Employee__c e : empList) {
            System.debug(
                'Emp ID: ' + e.Emp_ID__c +
                ', Name: ' + e.Name +
                ', Email: ' + e.Email__c +
                ', Birth Date: ' + e.Birth_Date__c +
                ', Department: ' + e.Department__c
            );
        }
    }

    public static void updateEmployee(Integer empId,
                                      String newDepartment) {

        Employee__c emp = [
            SELECT Id, Department__c
            FROM Employee__c
            WHERE Emp_ID__c = :empId
            LIMIT 1
        ];

        emp.Department__c = newDepartment;
        update emp;

        System.debug('Employee updated successfully');
    }

    public static void deleteEmployee(Integer empId) {

        Employee__c emp = [
            SELECT Id
            FROM Employee__c
            WHERE Emp_ID__c = :empId
            LIMIT 1
        ];

        delete emp;

        System.debug('Employee deleted successfully');
    }

```

```
}  
}
```

Save the class.

Step 4: Execute in Developer Console

Go to: **Developer Console** → **Debug** → **Open Execute Anonymous Window**

Step 5: Run Menu Operations

Create Employee

```
EmployeeSystem.menu(1);
```

View Employees

```
EmployeeSystem.menu(2);
```

Update Employee

```
EmployeeSystem.menu(3);
```

Delete Employee

```
EmployeeSystem.menu(4);
```

Viva Points

- Employee__c is a custom object.
- Name and Id are Salesforce default fields.
- SOQL fetches Salesforce records.
- DML operations used: insert, update, delete.
- Static methods allow direct calling.
- Execute Anonymous window acts like console execution.